CSI_6_SIT

Smart Internet Technologies

Abdul-Rahman Dimbil

S4112735

Table of Contents

Abstract

1. Introduction

2. Experimentation Set-up

2.1. Virtual machine environment

2.2. FFmpeg installation and media preparation

2.3. DASH manifest generation

2.4. Web hosting and unique URL access

2.5. Client playback using a free DASH player

2.6. iPerf traffic and traffic prioritisation

2.7. Traffic control scenarios

2.8. GitHub project organisation

3. Performance Evaluation

3.1. Normal Streaming Condition

3.2. TBF Scenario

3.3. HTB Scenario

3.4. Policing Scenario

3.5. Summary of MOS Findings

3.6. What the MOS Results Tell Us About QoE

3.7. How the End Device Can Improve QoE

4. Conclusion

References

Abstract

This report documents my end-to-end experiment setting up DASH video streaming using an Ubuntu server and a Windows client virtual machine. I started by installing FFmpeg to transcode two HD videos into three different bitrates (1.5, 2.0, and 4.0 Mbps) and generating the necessary DASH manifest files. The media was hosted on an Apache web server, and I used a free DASH player on the client side to access the streams via unique URLs. To test how the system behaves under different network conditions, I introduced iPerf traffic to compete with the video stream and used Linux traffic control to simulate three scenarios: TBF rate limiting, HTB bandwidth allocation, and ingress policing. I evaluated the viewing experience by assigning Mean Opinion Scores (MOS) to each scenario based on what I observed during playback. The results showed that HTB handled the limited bandwidth best, keeping playback reasonably smooth, while policing caused frequent buffering and made the video nearly unwatchable. This experiment gave me a practical understanding of how adaptive streaming responds to real-world network constraints and how different traffic management approaches affect what the user actually sees.

1. Introduction

Video streaming has become one of the most demanding applications on modern networks. Unlike sending an email or loading a webpage, streaming video needs a steady flow of data without too much delay or packet loss, otherwise the viewing experience falls apart. Dynamic Adaptive Streaming over HTTP, or DASH, is designed to handle exactly this problem. It lets the client switch between different quality versions of the same video depending on how much bandwidth is available at any given moment. If the network gets congested, the player drops to a lower bitrate to keep the video playing without buffering; when things clear up, it can jump back to a higher quality. This approach is what makes services like Netflix and YouTube work smoothly across all kinds of connections.

The reason I wanted to do this experiment was to see for myself how DASH behaves when the network isn't perfect. It's one thing to read about adaptive streaming in a textbook, but actually setting it up and then breaking it on purpose teaches you a lot more.

For this project, I built everything from scratch using virtual machines. I used FFmpeg to prepare the videos, Apache to host them, and a free DASH player on the client side to play them back. To test how the system copes under pressure, I used iPerf to generate background traffic and Linux traffic control to shape and police the connection. I looked at three specific scenarios: a simple bandwidth cap using TBF, a more flexible setup using HTB, and a policing rule that dropped packets once the rate went above a threshold. For each one, I watched the playback and assigned a Mean Opinion Score based on what I saw, following the standard ITU approach.

What this report covers is the whole process from start to finish. I walk through how I set everything up, the problems I ran into, and what I observed during testing. The focus is on how different network controls affect what the user actually experiences, rather than trying to invent a new algorithm or codec. I've kept things practical throughout.

The report is structured like this. Section 2 explains the setup, including the virtual machine environment, transcoding, DASH manifests, web hosting, and the traffic control configurations. Section 3 covers the performance evaluation, with the MOS results for each scenario and what they tell us about Quality of Experience. Section 4 wraps things up with a conclusion and some reflections on what I learned.

2. Experimentation Set-up

2.1. Virtual machine environment

I set up two virtual machines for this experiment using VirtualBox. One ran Ubuntu Server to host the media files, and the other ran Windows to act as the client. I configured the networking so both VMs could communicate with each other while the server could also access the internet to download packages and the test videos. This setup mimics how real streaming services work, where content sits on a server somewhere and viewers connect to it over the network. Keeping the roles separate also made it easier to troubleshoot when things went wrong.

```
vboxuser@ubuntu:~/Dashproject/videos$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:91:ce:96 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
       valid_lft 86321sec preferred_lft 86321sec
    inet6 fe80::e9bf:d22d:42ac:71d3/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:92:54:8a brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.9/24 brd 192.168.56.255 scope global dynamic noprefixroute enp0s8
       valid_lft 521sec preferred_lft 521sec
    inet6 fe80::fbc3:8683:f388:de97/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
vboxuser@ubuntu:~/Dashproject/videos$ ping 192.168.56.10
PING 192.168.56.10 (192.168.56.10) 56(84) bytes of data.
64 bytes from 192.168.56.10: icmp_seq=1 ttl=128 time=0.511 ms
64 bytes from 192.168.56.10: icmp_seq=2 ttl=128 time=0.385 ms
^C
--- 192.168.56.10 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1062ms
rtt min/avg/max/mdev = 0.385/0.448/0.511/0.063 ms
vboxuser@ubuntu:~/Dashproject/videos$
```

Server Configuration

```
Windows IP Configuration


Ethernet adapter Ethernet:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::f555:11d7:6a6f:a784%7
   IPv4 Address. . . . . . . . . . . : 10.0.2.15
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 10.0.2.2

Ethernet adapter Ethernet 2:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::900a:dbda:f5ea:aac7%5
   IPv4 Address. . . . . . . . . . . : 192.168.56.10
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . :

FLARE-VM 28/03/2026  6:53:20.76
C:\Users\munee\Desktop>
```

Client Machine

2.2. FFmpeg installation and media preparation

The first step on the server side was installing FFmpeg. I followed the standard installation process using apt, then verified it was working with the version command. Once FFmpeg was ready, I downloaded two HD videos from public sources and saved them to a working directory.

For each video, I needed to create multiple versions at different bitrates so the DASH player would have options to switch between depending on network conditions. I transcoded both videos to three bitrates: 1.5 Mbps, 2.0 Mbps, and 4.0 Mbps. The 4.0 Mbps version gave the best visual quality, while the lower bitrates were meant for when bandwidth was tight. Having these options is what makes adaptive streaming

possible – without them, the player would have no choice but to either buffer constantly or fail entirely when the network got congested.

```
vboxuser@ubuntu:~/Dashproject/videos$ ls
Video1.mp4  Video2.mp4
vboxuser@ubuntu:~/Dashproject/videos$ ffmpeg -i Video1.mp4 -c:v libx264 -b:v 1500k -preset fast -c:a aac Video1_15.mp4
ffmpeg version 3.4.11-0ubuntu0.1 Copyright (c) 2000-2022 the FFmpeg developers
  built with gcc 7 (Ubuntu 7.5.0-3ubuntu1~18.04)
  configuration: --prefix=/usr --extra-version=0ubuntu0.1 --toolchain=hardened --libdir=/usr/lib/x86_64-linux-gnu --incdir=/usr/include/x86_64-linux-gnu --enable-gpl --disable-stripping --enable-avresampl
e --enable-avisynth --enable-gnutls --enable-ladspa --enable-libass --enable-libbluray --enable-libbs2b --enable-libcaca --enable-libcdio --enable-libflite --enable-libfontconfig --enable-libfreetype --en
able-libfribidi --enable-libgme --enable-libgsm --enable-libmp3lame --enable-libmysofa --enable-libopenjpeg --enable-libopenmpt --enable-libopus --enable-libpulse --enable-librubberband --enable-librsvg -
-enable-libshine --enable-libsnappy --enable-libsoxr --enable-libspeex --enable-libssh --enable-libtheora --enable-libtwolame --enable-libvorbis --enable-libvpx --enable-libwavpack --enable-libwebp --enab
le-llbx265 --enable-libxml2 --enable-libxvid --enable-libzmq --enable-libzvbl --enable-omx --enable-openal --enable-opengl --enable-sdl2 --enable-libdc1394 --enable-libdrm --enable-libiec61883 --enable-ch
romaprint --enable-frei0r --enable-libopencv --enable-libx264 --enable-shared
  libavutil      55. 78.100 / 55. 78.100
  libavcodec     57.107.100 / 57.107.100
  libavformat    57. 83.100 / 57. 83.100
  libavdevice    57. 10.100 / 57. 10.100
  libavfilter     6.107.100 /  6.107.100
  libavresample   3.  7.  0 /  3.  7.  0
  libswscale      4.  8.100 /  4.  8.100
  libswresample   2.  9.100 /  2.  9.100
  libpostproc    54.  7.100 / 54.  7.100
Input #0, mov,mp4,m4a,3gp,3g2,mj2, from 'Video1.mp4':
  Metadata:
    major_brand     : mp42
    minor_version   : 0
    compatible_brands: mp42mp41
    creation_time   : 2018-10-27T11:51:02.000000Z
  Duration: 00:00:20.05, start: 0.000000, bitrate: 19209 kb/s
    Stream #0:0(eng): Video: h264 (High) (avc1 / 0x31637661), yuv420p(tv, bt709), 1920x1080 [SAR 1:1 DAR 16:9], 18933 kb/s, 25 fps, 25 tbr, 25k tbn, 50 tbc (default)
    Metadata:
      creation_time   : 2018-10-27T11:51:02.000000Z
      handler_name    : Alias Data Handler
      encoder         : AVC Coding
    Stream #0:1(eng): Audio: aac (LC) (mp4a / 0x6134706D), 48000 Hz, stereo, fltp, 317 kb/s (default)
    Metadata:
      creation_time   : 2018-10-27T11:51:02.000000Z
      handler_name    : Alias Data Handler
Stream mapping:
  Stream #0:0 -> #0:0 (h264 (native) -> h264 (libx264))
  Stream #0:1 -> #0:1 (aac (native) -> aac (native))
Press [q] to stop, [?] for help
[libx264 @ 0x560587416d00] using SAR=1/1
[libx264 @ 0x560587416d00] using cpu capabilities: MMX2 SSE2Fast SSSE3 SSE4.2 AVX BMI2
[libx264 @ 0x560587416d00] profile High, level 4.0
[libx264 @ 0x560587416d00] 264 - core 152 r2854 e9a5903 - H.264/MPEG-4 AVC codec - Copyleft 2003-2017 - http://www.videolan.org/x264.html - options: cabac=1 ref=2 deblock=1:0:0 analyse=0x3:0x113 me=hex su
bme=6 psy=1 psy_rd=1.00:0.00 mixed_ref=1 me_range=16 chroma_me=1 trellis=1 8x8dct=1 cqm=0 deadzone=21,11 fast_pskip=1 chroma_qp_offset=-2 threads=3 lookahead_threads=1 sliced_threads=0 nr=0 decimate=1 int
erlaced=0 bluray_compat=0 constrained_intra=0 bframes=3 b_pyramid=2 b_adapt=1 b_bias=0 direct=1 weightb=1 open_gop=0 weightp=1 keyint=250 keyint_min=25 scenecut=40 intra_refresh=0 rc_lookahead=30 rc=abr m
btree=1 bitrate=1500 ratetol=1.0 qcomp=0.60 qpmin=0 qpmax=69 qpstep=4 ip_ratio=1.40 aq=1:1.00
Output #0, mp4, to 'Video1_15.mp4':
```

Ffmpeg command running

```
vboxuser@ubuntu:~$ ffmpeg -version
ffmpeg version 3.4.11-0ubuntu0.1 Copyright (c) 2000-2022 the FFmpeg developers
  built with gcc 7 (Ubuntu 7.5.0-3ubuntu1~18.04)
  configuration: --prefix=/usr --extra-version=0ubuntu0.1 --toolchain=hardened --libdir=/usr/lib/x86_64-linux-gnu --incdir=/usr/include/x86_64-linux-gnu --enable-gpl --disable-stripping --enable-avresampl
e --enable-avisynth --enable-gnutls --enable-ladspa --enable-libass --enable-libbluray --enable-libbs2b --enable-libcaca --enable-libcdio --enable-libflite --enable-libfontconfig --enable-libfreetype --en
able-libfribidi --enable-libgme --enable-libgsm --enable-libmp3lame --enable-libmysofa --enable-libopenjpeg --enable-libopenmpt --enable-libopus --enable-libpulse --enable-librubberband --enable-librsvg -
-enable-libshine --enable-libsnappy --enable-libsoxr --enable-libspeex --enable-libssh --enable-libtheora --enable-libtwolame --enable-libvorbis --enable-libvpx --enable-libwavpack --enable-libwebp --enab
le-llbx265 --enable-libxml2 --enable-libxvid --enable-libzmq --enable-libzvbl --enable-omx --enable-openal --enable-opengl --enable-sdl2 --enable-libdc1394 --enable-libdrm --enable-libiec61883 --enable-ch
romaprint --enable-frei0r --enable-libopencv --enable-libx264 --enable-shared
  libavutil      55. 78.100 / 55. 78.100
  libavcodec     57.107.100 / 57.107.100
  libavformat    57. 83.100 / 57. 83.100
  libavdevice    57. 10.100 / 57. 10.100
  libavfilter     6.107.100 /  6.107.100
  libavresample   3.  7.  0 /  3.  7.  0
  libswscale      4.  8.100 /  4.  8.100
  libswresample   2.  9.100 /  2.  9.100
  libpostproc    54.  7.100 / 54.  7.100
Error splitting the argument list: Option not found
vboxuser@ubuntu:~$ systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/lib/systemd/system/apache2.service; enabled; vendor preset: enabled)
  Drop-in: /lib/systemd/system/apache2.service.d
           └─apache2-systemd.conf
   Active: active (running) since Wed 2026-03-25 22:27:40 PKT; 1min 59s ago
 Main PID: 3532 (apache2)
    Tasks: 55 (limit: 2326)
   CGroup: /system.slice/apache2.service
           ├─3532 /usr/sbin/apache2 -k start
           ├─3534 /usr/sbin/apache2 -k start
           └─3535 /usr/sbin/apache2 -k start
```

ffmeg version and apache service

```
vboxuser@ubuntu:~/Dashproject/videos$ ls
Video1.mp4  Video2.mp4
vboxuser@ubuntu:~/Dashproject/videos$ ffmpeg -i Video1.mp4 -c:v libx264 -b:v 1500k -preset fast -c:a aac Video1_15.mp4
ffmpeg version 3.4.11-0ubuntu0.1 Copyright (c) 2000-2022 the FFmpeg developers
  built with gcc 7 (Ubuntu 7.5.0-3ubuntu1~18.04)
  configuration: --prefix=/usr --extra-version=0ubuntu0.1 --toolchain=hardened --libdir=/usr/lib/x86_64-linux-gnu --incdir=/usr/include/x86_64-linux-gnu --enable-gpl --disable-stripping --enable-avresampl
e --enable-avisynth --enable-gnutls --enable-ladspa --enable-libass --enable-libbluray --enable-libbs2b --enable-libcaca --enable-libcdio --enable-libflite --enable-libfontconfig --enable-libfreetype --en
able-libfribidi --enable-libgme --enable-libgsm --enable-libmp3lame --enable-libmysofa --enable-libopenjpeg --enable-libopenmpt --enable-libopus --enable-libpulse --enable-librubberband --enable-librsvg -
-enable-libshine --enable-libsnappy --enable-libsoxr --enable-libspeex --enable-libssh --enable-libtheora --enable-libtwolame --enable-libvorbis --enable-libvpx --enable-libwavpack --enable-libwebp --enab
le-libx265 --enable-libxml2 --enable-libxvid --enable-libzmq --enable-libzvbl --enable-omx --enable-openal --enable-opengl --enable-sdl2 --enable-libdc1394 --enable-libdrm --enable-libiec61883 --enable-ch
romaprint --enable-frei0r --enable-libopencv --enable-libx264 --enable-shared
  libavutil      55. 78.100 / 55. 78.100
  libavcodec     57.107.100 / 57.107.100
  libavformat    57. 83.100 / 57. 83.100
  libavdevice    57. 10.100 / 57. 10.100
  libavfilter     6.107.100 /  6.107.100
  libavresample   3.  7.  0 /  3.  7.  0
  libswscale      4.  8.100 /  4.  8.100
  libswresample   2.  9.100 /  2.  9.100
  libpostproc    54.  7.100 / 54.  7.100
Input #0, mov,mp4,m4a,3gp,3g2,mj2, from 'Video1.mp4':
  Metadata:
    major_brand     : mp42
    minor_version   : 0
    compatible_brands: mp42mp41
    creation_time   : 2018-10-27T11:51:02.000000Z
  Duration: 00:00:20.05, start: 0.000000, bitrate: 19209 kb/s
    Stream #0:0(eng): Video: h264 (High) (avc1 / 0x31637661), yuv420p(tv, bt709), 1920x1080 [SAR 1:1 DAR 16:9], 18933 kb/s, 25 fps, 25 tbr, 25k tbn, 50 tbc (default)
    Metadata:
      creation_time   : 2018-10-27T11:51:02.000000Z
      handler_name    : Alias Data Handler
      encoder         : AVC Coding
    Stream #0:1(eng): Audio: aac (LC) (mp4a / 0x6134706D), 48000 Hz, stereo, fltp, 317 kb/s (default)
    Metadata:
      creation_time   : 2018-10-27T11:51:02.000000Z
      handler_name    : Alias Data Handler
Stream mapping:
  Stream #0:0 -> #0:0 (h264 (native) -> h264 (libx264))
  Stream #0:1 -> #0:1 (aac (native) -> aac (native))
Press [q] to stop, [?] for help
[libx264 @ 0x560587416d00] using SAR=1/1
[libx264 @ 0x560587416d00] using cpu capabilities: MMX2 SSE2Fast SSSE3 SSE4.2 AVX BMI2
[libx264 @ 0x560587416d00] profile High, level 4.0
[libx264 @ 0x560587416d00] 264 - core 152 r2854 e9a5903 - H.264/MPEG-4 AVC codec - Copyleft 2003-2017 - http://www.videolan.org/x264.html - options: cabac=1 ref=2 deblock=1:0:0 analyse=0x3:0x113 me=hex su
bme=6 psy=1 psy_rd=1.00:0.00 mixed_ref=1 me_range=16 chroma_me=1 trellis=1 8x8dct=1 cqm=0 deadzone=21,11 fast_pskip=1 chroma_qp_offset=-2 threads=3 lookahead_threads=1 sliced_threads=0 nr=0 decimate=1 int
erlaced=0 bluray_compat=0 constrained_intra=0 bframes=3 b_pyramid=2 b_adapt=1 b_bias=0 direct=1 weightb=1 open_gop=0 weightp=1 keyint=250 keyint_min=25 scenecut=40 intra_refresh=0 rc_lookahead=30 rc=abr m
btree=1 bitrate=1500 ratetol=1.0 qcomp=0.60 qpmin=0 qpmax=69 qpstep=4 ip_ratio=1.40 aq=1:1.00
Output #0, mp4, to 'Video1_15.mp4':
```

ffmpeg transcode the videos

```
vboxuser@ubuntu:~/Dashproject/videos$ ls -la
total 124828
drwxrwxr-x 2 vboxuser vboxuser     4096 جر ام  26 12:06 .
drwxrwxr-x 3 vboxuser vboxuser     4096 جر ام  25 22:45 ..
-rw-rw-r-- 1 vboxuser vboxuser  3699854 جر ام  26 10:44 Video1_15.mp4
-rw-rw-r-- 1 vboxuser vboxuser  4937764 جر ام  26 12:02 Video1_2.mp4
-rw-rw-r-- 1 vboxuser vboxuser  9888067 جر ام  26 12:03 Video1_4.mp4
-rw-rw-r-- 1 vboxuser vboxuser 48151385 جر ام  25 23:28 Video1.mp4
-rw-rw-r-- 1 vboxuser vboxuser  1711837 جر ام  26 12:05 Video2_15.mp4
-rw-rw-r-- 1 vboxuser vboxuser  2284199 جر ام  26 12:06 Video2_2.mp4
-rw-rw-r-- 1 vboxuser vboxuser  4614199 جر ام  26 12:07 Video2_4.mp4
-rw-rw-r-- 1 vboxuser vboxuser 52505441 جر ام  25 23:23 Video2.mp4
vboxuser@ubuntu:~/Dashproject/videos$
```

List of transcoded videos

2.3. DASH manifest generation

With the transcoded videos ready, I used FFmpeg again to generate the DASH manifests and segment the videos. This step creates two things: a manifest file (MPD) that tells the player what versions are available and how to request them, and a set of small video segments that the player can download individually. The segmentation is important because it lets the player switch quality on the fly – instead of downloading one huge file, it grabs small chunks and can change its mind between chunks if network conditions shift. I ran the FFmpeg command to produce the DASH output, making sure each video stream ended up in its own folder to keep things organised.

```
vboxuser@ubuntu:~/Dashproject/videos$ ffmpeg -i Video2_15.mp4 -i Video2_2.mp4 -i Video2_4.mp4 -map 0:v -map 1:v -map 2:v -map 0:a -c copy -f dash -segment_time 4 -adaptation_sets "id=0,streams=v id=1,str
ams=a" manifest2.mpd
ffmpeg version 3.4.11-0ubuntu0.1 Copyright (c) 2000-2022 the FFmpeg developers
  built with gcc 7 (Ubuntu 7.5.0-3ubuntu1~18.04)
  configuration: --prefix=/usr --extra-version=0ubuntu0.1 --toolchain=hardened --libdir=/usr/lib/x86_64-linux-gnu --incdir=/usr/include/x86_64-linux-gnu --enable-gpl --disable-stripping --enable-avresamp
e --enable-avisynth --enable-gnutls --enable-ladspa --enable-libass --enable-libbluray --enable-libbs2b --enable-libcaca --enable-libcdio --enable-libflite --enable-libfontconfig --enable-libfreetype --e
able-libfribidi --enable-libgme --enable-libgsm --enable-libmp3lame --enable-libmysofa --enable-libopenjpeg --enable-libopenmpt --enable-libopus --enable-libpulse --enable-librubberband --enable-librsvg
-enable-libshine --enable-libsnappy --enable-libsoxr --enable-libspeex --enable-libssh --enable-libtheora --enable-libtwolame --enable-libvorbis --enable-libvpx --enable-libwavpack --enable-libwebp --ena
le-libx265 --enable-libxml2 --enable-libxvid --enable-libzmq --enable-libzvbi --enable-omx --enable-openal --enable-opengl --enable-sdl2 --enable-libdc1394 --enable-libdrm --enable-libiec61883 --enable-c
romaprint --enable-frei0r --enable-libopencv --enable-libx264 --enable-shared
  libavutil      55. 78.100 / 55. 78.100
  libavcodec     57.107.100 / 57.107.100
  libavformat    57. 83.100 / 57. 83.100
  libavdevice    57. 10.100 / 57. 10.100
  libavfilter     6.107.100 /  6.107.100
  libavresample   3.  7.  0 /  3.  7.  0
  libswscale      4.  8.100 /  4.  8.100
  libswresample   2.  9.100 /  2.  9.100
  libpostproc    54.  7.100 / 54.  7.100
Input #0, mov,mp4,m4a,3gp,3g2,mj2, from 'Video2_15.mp4':
  Metadata:
    major_brand     : isom
    minor_version   : 512
    compatible_brands: isomiso2avc1mp41
    encoder         : Lavf57.83.100
  Duration: 00:00:10.07, start: 0.000000, bitrate: 1359 kb/s
    Stream #0:0(eng): Video: h264 (High) (avc1 / 0x31637661), yuv420p, 3840x2160 [SAR 1:1 DAR 16:9], 1352 kb/s, 30 fps, 30 tbr, 15360 tbn, 60 tbc (default)
    Metadata:
      handler_name    : VideoHandler
    Stream #0:1(eng): Audio: aac (LC) (mp4a / 0x6134706D), 48000 Hz, stereo, fltp, 2 kb/s (default)
    Metadata:
      handler_name    : SoundHandler
Input #1, mov,mp4,m4a,3gp,3g2,mj2, from 'Video2_2.mp4':
  Metadata:
    major_brand     : isom
    minor_version   : 512
    compatible_brands: isomiso2avc1mp41
    encoder         : Lavf57.83.100
  Duration: 00:00:10.07, start: 0.000000, bitrate: 1814 kb/s
    Stream #1:0(eng): Video: h264 (High) (avc1 / 0x31637661), yuv420p, 3840x2160 [SAR 1:1 DAR 16:9], 1809 kb/s, 30 fps, 30 tbr, 15360 tbn, 60 tbc (default)
    Metadata:
      handler_name    : VideoHandler
```

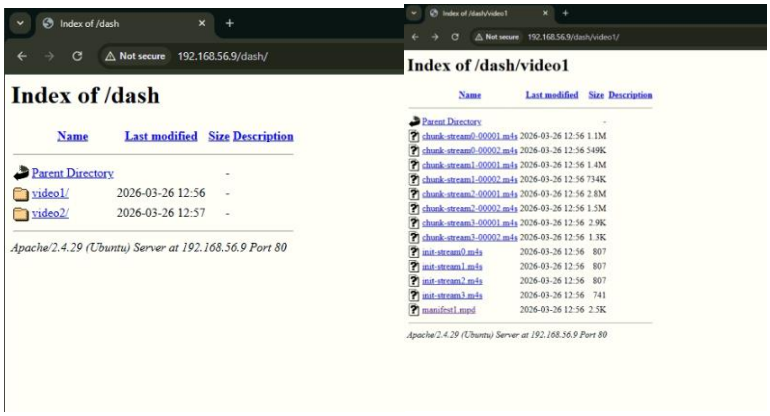
Manifest the videos to different Mbps

```
vboxuser@ubuntu:~/Dashproject/videos$ ls -la
total 133248
drwxrwxr-x 2 vboxuser vboxuser     4096 جر ام  26 12:47 .
drwxrwxr-x 3 vboxuser vboxuser     4096 جر ام  25 22:45 ..
-rw-rw-r-- 1 vboxuser vboxuser  1137857 جر ام  26 12:47 chunk-stream0-00001.m4s
-rw-rw-r-- 1 vboxuser vboxuser   561760 جر ام  26 12:47 chunk-stream0-00002.m4s
-rw-rw-r-- 1 vboxuser vboxuser  1520285 جر ام  26 12:47 chunk-stream1-00001.m4s
-rw-rw-r-- 1 vboxuser vboxuser   751694 جر ام  26 12:47 chunk-stream1-00002.m4s
-rw-rw-r-- 1 vboxuser vboxuser  2978566 جر ام  26 12:47 chunk-stream2-00001.m4s
-rw-rw-r-- 1 vboxuser vboxuser  1623413 جر ام  26 12:47 chunk-stream2-00002.m4s
-rw-rw-r-- 1 vboxuser vboxuser     3020 جر ام  26 12:47 chunk-stream3-00001.m4s
-rw-rw-r-- 1 vboxuser vboxuser     1336 جر ام  26 12:47 chunk-stream3-00002.m4s
-rw-rw-r-- 1 vboxuser vboxuser      807 جر ام  26 12:47 init-stream0.m4s
-rw-rw-r-- 1 vboxuser vboxuser      807 جر ام  26 12:47 init-stream1.m4s
-rw-rw-r-- 1 vboxuser vboxuser      807 جر ام  26 12:47 init-stream2.m4s
-rw-rw-r-- 1 vboxuser vboxuser      741 جر ام  26 12:47 init-stream3.m4s
-rw-rw-r-- 1 vboxuser vboxuser     2611 جر ام  26 12:44 manifest1.mpd
-rw-rw-r-- 1 vboxuser vboxuser     2656 جر ام  26 12:47 manifest2.mpd
-rw-rw-r-- 1 vboxuser vboxuser  3699854 جر ام  26 10:44 Video1_15.mp4
-rw-rw-r-- 1 vboxuser vboxuser  4937764 جر ام  26 12:02 Video1_2.mp4
-rw-rw-r-- 1 vboxuser vboxuser  9888067 جر ام  26 12:03 Video1_4.mp4
-rw-rw-r-- 1 vboxuser vboxuser 48151385 جر ام  25 23:28 Video1.mp4
-rw-rw-r-- 1 vboxuser vboxuser  1711837 جر ام  26 12:05 Video2_15.mp4
-rw-rw-r-- 1 vboxuser vboxuser  2284199 جر ام  26 12:06 Video2_2.mp4
-rw-rw-r-- 1 vboxuser vboxuser  4614199 جر ام  26 12:07 Video2_4.mp4
-rw-rw-r-- 1 vboxuser vboxuser 52505441 جر ام  25 23:23 Video2.mp4
```

DASH Manifest videos

2.4. Web hosting and unique URL access

To make the streams accessible, I installed Apache on the Ubuntu server and moved the DASH folders into the web root directory. I gave each stream its own URL so the client could point directly to the manifest file. This part was surprisingly simple – once Apache was running, the files were just there on the network. I tested the URLs from the server itself first using curl, then later from the client VM to confirm everything was reachable.



Webservice and available videos

2.5. Client playback using a free DASH player

On the Windows client VM, I opened a browser and used an open-source DASH reference player. I entered the full URL to one of the manifest files and hit play. The video started streaming within a few seconds, which told me the whole pipeline was working – FFmpeg had done its job, Apache was serving the files correctly, and the player could interpret the manifest and download the segments. It was a good feeling seeing it actually work after spending time setting everything up.

DASH player showing video playback on client

2.6. iPerf traffic and traffic prioritisation

The next phase was introducing competition for bandwidth. I set up iPerf3 to generate a constant TCP stream from the client to the server at 1 Mbps. This simulated other traffic on the network alongside the video stream. The idea was to see how the DASH player coped when it had to share bandwidth with something else.

I then used Linux traffic control to give the iPerf traffic higher priority than the video stream. The goal was to push the video into a lower priority class so that when bandwidth got tight, the iPerf flow would get preference. This let me test how the DASH player reacts when it's not the most important thing on the network – which is realistic for many scenarios where video is just one of several applications running at the same time.

```
vboxuser@ubuntu:~/Dashproject/videos$ iperf3 -s
-----------------------------------------------------------
Server listening on 5201
-----------------------------------------------------------
Accepted connection from 192.168.56.10, port 49899
[  5] local 192.168.56.9 port 5201 connected to 192.168.56.10 port 49900
[ ID] Interval           Transfer     Bandwidth
[  5]   0.00-1.00   sec   115 KBytes   941 Kbits/sec
[  5]   1.00-2.00   sec   121 KBytes   992 Kbits/sec
[  5]   2.00-3.00   sec   123 KBytes  1.01 Mbits/sec
[  5]   3.00-4.00   sec   114 KBytes   934 Kbits/sec
[  5]   4.00-5.00   sec   129 KBytes  1.06 Mbits/sec
[  5]   5.00-6.00   sec   122 KBytes  1000 Kbits/sec
[  5]   6.00-7.01   sec  92.0 KBytes   750 Kbits/sec
[  5]   7.01-8.00   sec   154 KBytes  1.27 Mbits/sec
[  5]   8.00-9.00   sec   120 KBytes   985 Kbits/sec
[  5]   9.00-10.00  sec   124 KBytes  1.02 Mbits/sec
[  5]  10.00-10.06  sec  8.00 KBytes  1.06 Mbits/sec
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bandwidth
[  5]   0.00-10.06  sec  0.00 Bytes  0.00 bits/sec                  sender
[  5]   0.00-10.06  sec  1.19 MBytes   995 Kbits/sec                  receiver
-----------------------------------------------------------
```

Iperf3 TCP traffic at 1M

```
vboxuser@ubuntu:~/Dashproject/videos$ tc -s class show dev enp0s8
class htb 1:10 root prio rate 1Mbit ceil 5Mbit burst 1600b cburst 1600b
 Sent 270 bytes 5 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 5 borrowed: 0 giants: 0
 tokens: 193250 ctokens: 38650

class htb 1:20 root prio rate 1Mbit ceil 5Mbit burst 1600b cburst 1600b
 Sent 852 bytes 6 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 6 borrowed: 0 giants: 0
 tokens: 194750 ctokens: 38950

vboxuser@ubuntu:~/Dashproject/videos$ tc -s class show dev enp0s8
class htb 1:10 root prio rate 1Mbit ceil 5Mbit burst 1600b cburst 1600b
 Sent 540 bytes 10 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 10 borrowed: 0 giants: 0
 tokens: 193250 ctokens: 38650

class htb 1:20 root prio rate 1Mbit ceil 5Mbit burst 1600b cburst 1600b
 Sent 852 bytes 6 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 6 borrowed: 0 giants: 0
 tokens: 194750 ctokens: 38950

vboxuser@ubuntu:~/Dashproject/videos$ tc -s class show dev enp0s8
class htb 1:10 root prio rate 1Mbit ceil 5Mbit burst 1600b cburst 1600b
 Sent 540 bytes 10 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 10 borrowed: 0 giants: 0
 tokens: 193250 ctokens: 38650

class htb 1:20 root prio rate 1Mbit ceil 5Mbit burst 1600b cburst 1600b
 Sent 852 bytes 6 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 6 borrowed: 0 giants: 0
 tokens: 194750 ctokens: 38950
```

Implementation of Priority Rule

2.7. Traffic control scenarios

I set up different network impairment scenarios to see how each affected video playback.

TBF (Token Bucket Filter): I applied a simple rate limit of 2.5 Mbps on the server's egress interface. I set the burst to 20 KB and configured the latency limit to 50 ms. This created a hard ceiling on how fast video data could leave the server.

```
vboxuser@ubuntu:~/Dashproject/videos$ tc -s qdisc show dev enp0s8
qdisc tbf 8001: root refcnt 2 rate 2500Kbit burst 20Kb lat 50.0ms
 Sent 738 bytes 9 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
vboxuser@ubuntu:~/Dashproject/videos$ tc -s qdisc show dev enp0s8
qdisc tbf 8001: root refcnt 2 rate 2500Kbit burst 20Kb lat 50.0ms
 Sent 1008 bytes 14 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
vboxuser@ubuntu:~/Dashproject/videos$ A
A: command not found
vboxuser@ubuntu:~/Dashproject/videos$ tc -s qdisc show dev enp0s8
qdisc tbf 8001: root refcnt 2 rate 2500Kbit burst 20Kb lat 50.0ms
 Sent 1092 bytes 16 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
```

Token Bucket Filter at 2.5 MBPS

HTB (Hierarchical Token Bucket): This was more flexible. I set up a class with a guaranteed minimum of 2.5 Mbps but allowed it to use up to 5 Mbps if bandwidth was available. The idea was to let the video burst higher when possible while still protecting the iPerf traffic.

```
vboxuser@ubuntu:~/Dashproject/videos$ tc -s class show dev enp0s8
class htb 1:10 root prio rate 2500Kbit ceil 5Mbit burst 20Kb cburst 1600b
 Sent 354 bytes 7 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 7 borrowed: 0 giants: 0
 tokens: 1021900 ctokens: 38950
```

Hierarchical Token Bucket

Traffic Policing: For the third scenario, I applied policing on the client's ingress interface. Instead of delaying packets like shaping does, policing drops any traffic that exceeds 3.5 Mbps. This is a more aggressive form of control and tends to cause packet loss rather than just delay.

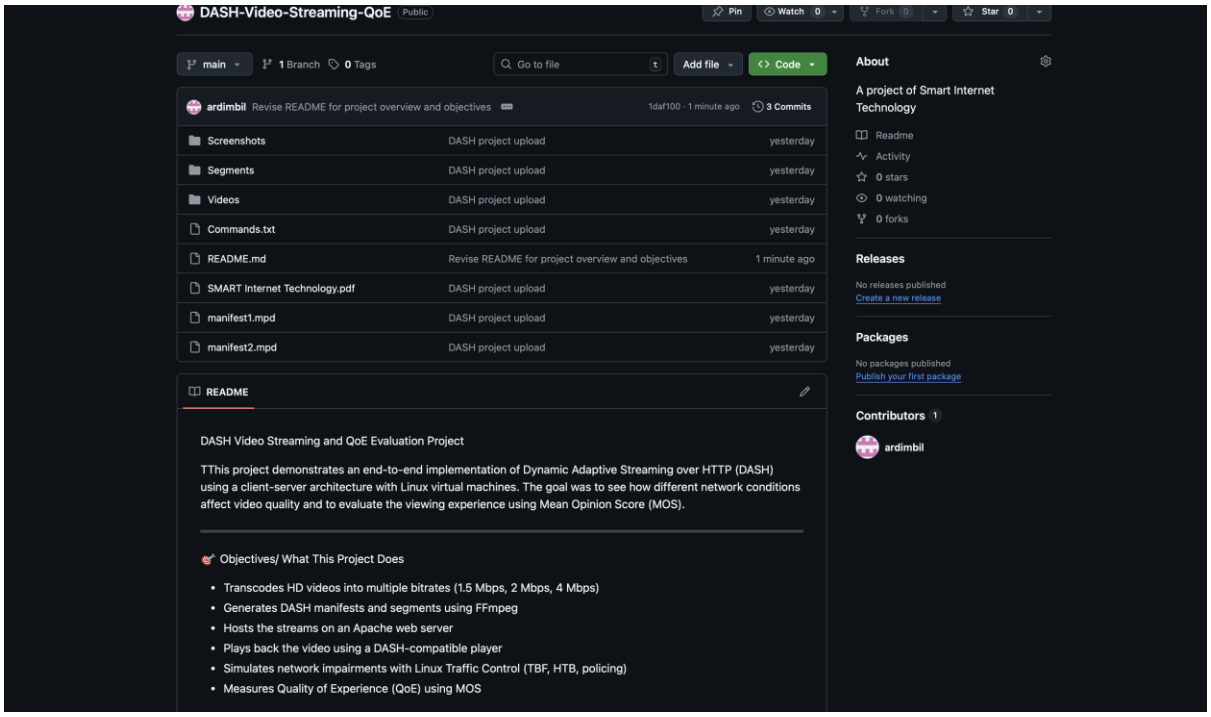
```
vboxuser@ubuntu:~/Dashproject/videos$ tc -s class show dev enp0s8
class htb 1:10 root prio rate 3500Kbit ceil 3500Kbit burst 20Kb cburst 1599b
 Sent 0 bytes 0 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 0 borrowed: 0 giants: 0
 tokens: 731421 ctokens: 57140

vboxuser@ubuntu:~/Dashproject/videos$ tc -s class show dev enp0s8
class htb 1:10 root prio rate 3500Kbit ceil 3500Kbit burst 20Kb cburst 1599b
 Sent 270 bytes 5 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 5 borrowed: 0 giants: 0
 tokens: 729492 ctokens: 55211

vboxuser@ubuntu:~/Dashproject/videos$ tc -s class show dev enp0s8
class htb 1:10 root prio rate 3500Kbit ceil 3500Kbit burst 20Kb cburst 1599b
 Sent 354 bytes 7 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 7 borrowed: 0 giants: 0
 tokens: 729921 ctokens: 55640

vboxuser@ubuntu:~/Dashproject/videos$ tc -s class show dev enp0s8
class htb 1:10 root prio rate 3500Kbit ceil 3500Kbit burst 20Kb cburst 1599b
 Sent 354 bytes 7 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 7 borrowed: 0 giants: 0
 tokens: 729921 ctokens: 55640

vboxuser@ubuntu:~/Dashproject/videos$ tc -s class show dev enp0s8
class htb 1:10 root prio rate 3500Kbit ceil 3500Kbit burst 20Kb cburst 1599b
 Sent 354 bytes 7 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 7 borrowed: 0 giants: 0
 tokens: 729921 ctokens: 55640

vboxuser@ubuntu:~/Dashproject/videos$ tc -s class show dev enp0s8
class htb 1:10 root prio rate 3500Kbit ceil 3500Kbit burst 20Kb cburst 1599b
 Sent 354 bytes 7 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 7 borrowed: 0 giants: 0
 tokens: 729921 ctokens: 55640
```

Traffic Policing in the ingress

2.8. GitHub project organisation

I uploaded everything to GitHub to keep the project organised and make it easy to share. The repository includes folders for the original videos, transcoded versions, DASH manifests, screenshots, and configuration scripts. I also wrote a README file with instructions on how to set everything up from scratch. Having everything in one place makes it easier to revisit later or share with someone else who wants to try the same experiment.

GitHub repository: https://github.com/ardimbil/DASH-Video-Streaming-QoE



3. Performance Evaluation

For each network scenario, I watched the video playback carefully and noted what I saw. Based on those observations, I assigned a Mean Opinion Score (MOS) following the ITU-T BT.500 approach. This is the standard way to turn subjective viewing experiences into numbers that can be compared. I looked at things like how often buffering happened, whether the quality kept changing, and whether the video ever completely stalled.

3.1. Normal Streaming Condition

Before applying any traffic controls, I tested the system with no network limitations. The DASH player immediately picked the highest bitrate (4 Mbps) and stayed there throughout. The video started quickl and played without any buffering or stuttering. The quality was consistent and looked good. Based on this, I gave it an MOS of 5 (Excellent). This was my baseline to compare everything else against.

3.2. TBF Scenario

With the TBF limit set to 2.5 Mbps, the player could no longer sustain the 4 Mbps stream. It dropped down to lower bitrates prettyquickly. I noticed occasional buffering, especially when the player tried to switch back to a higher quality. The video was still watchable, but the constant quality changes were distracting. I ended up giving this scenrio an MOS of 3.5 (Fair to Good). The hard cap on bandwidth meant the player had to compromise, and it showed.

3.3. HTB Scenario

HTB performed noticeably better than TBF. Even though the guaranteed rate was also 2.5 Mbps, the player could burst higher when the network allowed it. This meant I saw periods of better quality mixed in with the lower bitrate segments. Buffering was less frequent, and playback felt smoother overall. I gave this an MOS of 4 (Good). The flexibility of HTB seemed to give the adaptive algorithm more room to work with.

3.4. Policing Scenario

Policing was the worst by far. As soon as the traffic exceeded 3.5 Mbps, packets started getting dropped. The video player didn't handle this well at all – it kept stalling, the quality dropped to the lowest setting, and at times the playback just paused completely. Unlike shaping which just slows things down, dropping packets makes the player think the network has failed entirely. I rated this scenario as MOS 2 (Poor). It was barely watchable

3.5. Summary of MOS Findings

Scenario	What I observed	MOS
Normal	Smooth playback, no buffering, high quality	5
TBF	Reduced quality, occasional buffering, visible switches	3.5
HTB	Mostly smooth, less buffering, some quaality variation	4
Policing	Frequent stalling, cosnstant buffering, very poor experience	2

Looking at these results, a few things stood out to me. First, bandwidth limiting alone (TBF) made the experience worse, but it was still acceptable. The player adapted and kept the video playing, even if quality took a hit. That's the whole point of DASH – it's designed to degrade gracefully rather than fail completely.

HTB gave the best balance. The ability to burst above the guaranteed rate meant the player could grab higher quality segments when bandwidth was available, which made the experience feel less constrained. This matches what other researchers have found about hierarchical shaping being more suitable for adaptive streaming [1].

Policing was a different story. Dropping packets instead of delaying them caused real problems. The player's TCP connections would time out, segments wouldn't arrive, and the video would just freeze. This shows that how you control traffic matters just as much as how much you limit it. Shaping (delaying) gives the adaptive algorithm a chance to respond; policing (dropping) just breaks things [2].

What I took away from this is that QoE isn't just about bandwidth numbers. The way the network treats packets – whether it delays them or drops them – has a huge impact on what the user actually sees. Even with the same average throughput, a shaped connection feels smoother than one where packets are being dropped [3].

3.7. How the End Device Can Improve QoE

Even with a perfect network, the client device itself plays a big role in how good the experience feels. Here are a few things I noticed that make a difference:

Buffer size: A larger playback buffer helps smooth out small network hiccups. If the player has a few seconds of video stored up, a short period of low bandwidth won't cause buffering. The trade-off is that the stream takes longer to start, but during playback it feels more stable.

Adaptive bitrate logic: Not all DASH players are the same. Some switch quality too aggressively, which causes constant visual changes that are annoying. Others are too slow to react and end up buffering. A well-tuned algorithm finds the right balance [4].

Decoding performance: The device needs to actually handle the video it's being sent. If the CPU or GPU can't keep up with high bitrate streams, the video will drop frames or stutter regardless of network conditions. This was less of an issue in my VM environment, but on real devices it matters.

Network interface: A stable Wi-Fi or wired connection makes a difference. Background traffic on the client side – like other apps using the network – can also affect streaming performance. Closing unnecessary apps helps.

TCP settings: Some modern TCP congestion control algorithms handle variable networks better than others. This is more of a system-level thing than something the average user can tweak, but it still affects the experience [5].

4. Conclusion

This experiment gave me a much better understanding of how DASH streaming actually works under the hood. Setting up the whole pipeline from scratch – transcoding videos, generating manifests, hosting them, and then breaking the network on purpose – showed me how each piece fits together. The MOS results confirmed what I expected: bandwidth limits hurt quality, but policing hurts even more because packet loss breaks the player's ability to adapt. HTB turned out to be the most user-friendly approach, letting the stream burst higher when possible while still keeping things fair.

I also learned that QoE depends on more than just network conditions. The client device, the player software, and even things like buffer settings all play a role. If I were to do this again, I'd probably try more complex traffic patterns or test different DASH players to see how they compare. Overall, this was a useful hands-on exercise that connected what I've been studying in lectures to something I could actually build and break myself.

References

[1] D. Tanjung, J.-D. Kim, D.-H. Kim, J. Lee, S. Kim and J.-Y. Jung, "QoE Optimization in DASH-Based Multiview Video Streaming," *IEEE Access*, vol. 11, pp. 83603–83614, 2023.

[2] D. Z. Rodríguez, Z. Wang, R. L. Rosa and G. Bressan, "The impact of video-quality-level switching on user quality of experience in dynamic adaptive streaming over HTTP," *Journal of Wireless Communications and Networking*, 2014.

[3] L. Du, L. Zhuo, J. Li, J. Zhang, X. Li and H. Zhang, "Video Quality of Experience Metric for Dynamic Adaptive Streaming Services Using DASH Standard and Deep Spatial-Temporal Representation of Video," *Applied Sciences*, vol. 10, p. 5, 2020.

[4] S. K. M. Mehr, P. Jogalekar and D. Medhi, "Moving QoE for monitoring DASH video streaming: models and a study of multiple mobile clients," *Journal of Internet Services and Applications*, vol. 12, p. 1, 2021.

[5] Q. Wang, H.-N. Dai, D. Wu and H. Xiao, "Data analysis on video streaming QoE over mobile networks," *EURASIP Journal on Wireless Communications and Networking*, 2018.

AI Declaration

AI tools (ChatGPT) were used to assist with grammar checking, structure refinement, and language polishing for this report. All technical implementation, experimental work, observations, and analysis are my own.